



SVILUPPATORE



CLIENTE

DOCUMENTAZIONE TECNICA DI SISTEMA

ESG Nexus

Architettura · Sicurezza · Flussi Operativi

Questo documento descrive l'architettura tecnica, il modello dei dati, i meccanismi di sicurezza e i flussi operativi del sistema così come è stato progettato e realizzato. È destinato a chi necessita di una comprensione tecnica precisa del sistema.

PRODOTTO ESG Nexus — Applicazione web per la consulenza di sostenibilità

VERSIONE DOCUMENTO 1.0 — Aprile 2026

SVILUPPATORE axialoop

CLIENTE Evalis

STANDARD SUPPORTATI GRI · CSRD/ESRS · Entrambi

CLASSIFICAZIONE Riservato — Solo uso interno

Introduzione

ESG Nexus è un'applicazione web gestionale per la consulenza di sostenibilità. Permette a un consulente ESG di organizzare e condurre l'intero processo di rendicontazione per i propri clienti aziendali — dalla fase di acquisizione e pre-qualifica fino alla chiusura e pubblicazione del bilancio di sostenibilità — seguendo gli standard GRI, CSRD/ESRS o entrambi.

Il sistema gestisce clienti, engagement (progetti di rendicontazione), moduli di raccolta dati operativi, calcolo delle emissioni GHG, KPI ambientali/sociali/governance, registro rischi, scadenze, milestone, fatturazioni e audit trail. I dati sono strutturati intorno a un ciclo procedurale standard in 8 fasi (PROC-00 → PROC-07), per un totale di 61 moduli di raccolta dati per engagement.

Scopo di Questo Documento

Questo documento descrive l'architettura tecnica, il modello dei dati, i meccanismi di sicurezza e i flussi operativi del sistema così come è stato progettato e realizzato. È destinato a chi necessita di una comprensione tecnica precisa del sistema: come sono strutturati i dati, come viene garantita la sicurezza e l'isolamento tra utenti, come funzionano i processi automatizzati in background, e su quale infrastruttura il sistema opera.

Non è un manuale utente né una guida operativa. È una documentazione tecnica di riferimento sull'implementazione.

Engagement

Ogni cliente che avvia un percorso di rendicontazione ESG genera un **Engagement**: un progetto strutturato identificato da un codice univoco del tipo `ESG-2026-ACM-001` (anno, sigla cliente, numero progressivo), che percorre 8 fasi operative standard (PROC-00 → PROC-07), ciascuna articolata in moduli di raccolta dati (da 7 a 9 form per fase). Al termine del processo, tutti i dati sono disponibili per la generazione del bilancio di sostenibilità.

Indice

§	SEZIONE	Pag.
01	Introduzione & Engagement	2
02	Stack Tecnologico & Architettura Logica	3
03	Il Motore Operativo: PROC-00 → PROC-07	4
04	Database: Progettazione e Integrità dei Dati	5
05	Sicurezza: Architettura a Difesa Stratificata	7
06	Automazione: Edge Functions e Trigger	9
07	Persistenza dei Dati del Form	10
08	Analytics e Business Intelligence	11
09	Dashboard: Situazione in Tempo Reale	12
10	Gestione Rischi: Il Registro Integrato	13

11	Resilienza e Continuità del Dato	13
12	Qualità del Codice e Processo di Sviluppo	14
13	Funzionalità di Stampa e Archiviazione	15
14	Glossario Tecnico	16
15	Evoluzioni Future del Sistema	17

§02

Il Cuore Tecnologico: Architettura del Sistema

Stack Tecnologico

ESG Nexus è costruito su tecnologie enterprise-grade, tutte nella loro versione stabile più recente:

LAYER	TECNOLOGIA	VERSIONE	RUOLO
Frontend	React	18.2	UI reattiva, single-page application
Build tool	Vite	6.1	Bundle ottimizzato, hot reload, code splitting
Routing	React Router	6.26	Navigazione client-side, lazy loading
State/Cache	TanStack Query	5.100	Cache server-state, sincronizzazione, deduplication
UI Components	shadcn/ui + Tailwind	3.4	Design system coerente, accessibile
Grafici	Recharts	2.15	Visualizzazioni dati interattive
Validazione	Zod	3.25	Schema-first data validation
Backend/DB	Supabase (PostgreSQL 15)	—	Database relazionale, Auth, Realtime, Storage
Edge Functions	Deno runtime	—	Server-side logic serverless
Deployment	Vercel	—	CDN globale, HTTPS automatico, preview deployments
CI/CD	GitHub Actions	—	Lint + test + build automatici su ogni push

Architettura Logica

Il sistema adotta un'architettura **"Backend-as-a-Service First"**: tutta la logica di persistenza, sicurezza e automazione vive nel layer Supabase/PostgreSQL. Il frontend React si comporta come un client intelligente — non contiene logica di business critica, non gestisce segreti, non persiste dati localmente. Questa separazione garantisce che, anche in caso di vulnerabilità nel codice frontend, i dati rimangano protetti dalle policy di sicurezza a livello database.

Browser (React SPA)

|

| HTTPS + JWT Auth

▼

Supabase API Layer

├─ PostgREST (REST → SQL, RLS enforced)

├─ Auth (JWT, sessioni, OAuth)

├─ Realtime (WebSocket, postgres_changes)

├─ Storage (bucket S3-compatible, RLS)

└─ Edge Functions (Deno, serverless)

|

▼

PostgreSQL 15 (managed)

18 tabelle · 8 migrazioni · 4 Views · 2 trigger functions · pg_cron

TABELLE DATABASE

18

PostgreSQL 15 managed · 8 migrazioni

FORM PER ENGAGEMENT

61

PROC-00 → PROC-07 · 8 procedure operative

§03

Il Motore Operativo: PROC-00 → PROC-07

■

Il Flusso dell'Engagement

Ogni engagement percorre un ciclo di vita strutturato in 8 procedure operative (PROC), ciascuna corrispondente a una fase distinta del processo di consulenza ESG. Il progresso complessivo dell'engagement è calcolato automaticamente dal sistema come media pesata del completamento di tutti i form.

PROCEDURA	FASE	N° FORM	CONTENUTO
PROC-00	Acquisizione Cliente	7	Primo contatto, pre-qualifica, score, conflitti di interesse, offerta commerciale, KYC, chiusura fase
PROC-01	Analisi Contesto	7	Contesto organizzativo, catena del valore, stakeholder
PROC-02	Analisi di Materialità	7	IRO identification, scoring materialità, prioritizzazione
PROC-03	Raccolta Dati ESG	7	Dati ambientali, sociali, governance
PROC-04	Calcolo GHG	7	Emissioni Scope 1/2/3, fattori di emissione, tCO ₂ e
PROC-05	Validazione KPI	7	KPI E/S/G, valori attuale vs target, gap analysis
PROC-06	Redazione Bilancio	9	Capitoli bilancio, revisioni, approvazioni
PROC-07	Chiusura & Pubblicazione	8	Audit finale, log attività, archiviazione
TOTALE			
61 form di raccolta dati per engagement, ognuno con persistenza automatica su Supabase.			

■

La Logica di Progresso Deterministico

Il progresso di ogni procedura non è un dato inserito manualmente: viene calcolato automaticamente da una Edge Function (`compute-engagement-progress`) ogni volta che un form viene salvato. L'algoritmo è deterministico e privo di ambiguità:

- Un form **completato** contribuisce al 100% del suo peso
- Un form **in corso** contribuisce al 50% del suo peso
- La percentuale di fase = `(completati + in_corso × 0.5) / totale_form × 100`
- Il progresso globale dell'engagement = **media aritmetica** delle percentuali di tutte le 8 procedure

Questo significa che il consulente vede sempre una percentuale di avanzamento che riflette la realtà effettiva del lavoro svolto, calcolata server-side, non manipolabile dalla UI.

§04

Database: Progettazione e Integrità dei Dati

■

Schema Relazionale (18 Tabelle)

Il database è strutturato in tre livelli gerarchici:

LIVELLO 1 — ENTITÀ PRINCIPALI (CON USER_ID)

- `users_profile` — profilo consulente (linked ad auth.users)
- `clienti` — anagrafica cliente (partita IVA EU fino a 16 char, codice ISO nazione, validazione ATECO)
- `engagements` — progetto ESG (codice univoco, standard, stato, budget)
- `rischi` — registro rischi (score calcolato come colonna `GENERATED ALWAYS AS probabilita * impatto`)
- `azioni_giorno` — task list operativa

LIVELLO 2 — DATI DI ENGAGEMENT (FIGLI DI ENGAGEMENTS)

- `engagement_fasi` — 8 righe per engagement, PROC-00→PROC-07 (UNIQUE constraint su engagement+proc_code)
- `form_data` — contenuto JSONB dei form (UNIQUE su engagement+form_code)
- `iro_engagement` — IRO selezionati per engagement (linked al catalogo)
- `ghg_voci` — emissioni GHG (`ROUND((quantita * fattore_emissione / 1000)::numeric, 4)` STORED)
- `kpi_valori` — KPI misurati (UNIQUE su engagement+kpi_code+anno)
- `scadenze` — deadline con stato e priorità (indice composito su data+stato)
- `milestone` — pietre miliari del progetto
- `capitoli_bilancio` — sezioni del bilancio (stato: bozza → in_revisione → approvato)
- `fatturazioni` — fatture in centesimi di euro (evita arrotondamenti floating-point)
- `eventi_log` — audit trail completo

LIVELLO 3 — CATALOGHI (READ-ONLY, CONDIVISI)

- `catalogo_iro` — 24 Impact, Risk & Opportunity standard pre-caricati (E/S/G, punteggi materialità)
- `kpi_definizioni` — 19 KPI standard predefiniti con framework di riferimento (GRI, CSRD_ESRS)
- `contatti_cliente` — rubrica contatti per cliente

■

Colonne Calcolate: Integrità Matematica Garantita dal DB

Due misurazioni critiche sono definite come colonne `GENERATED ALWAYS AS STORED`, il che significa che il valore è calcolato e scritto da PostgreSQL al momento dell'INSERT/UPDATE, non è mai passato dal frontend:

Score del rischio:

```
score integer GENERATED ALWAYS AS (probabilita * impatto) STORED
```

La matrice di rischio 5×5 (probabilità × impatto, range 1–25) è garantita matematicamente consistente. Nessun utente può inserire manualmente uno score errato.

Emissioni GHG in tonnellate CO₂ equivalente:

```
co2e_tonnellate numeric(15,4) GENERATED ALWAYS AS (  
    CASE WHEN quantita IS NOT NULL AND fattore_emissione IS NOT NULL  
        THEN ROUND((quantita * fattore_emissione / 1000)::numeric, 4)  
        ELSE NULL END  
) STORED
```

Il calcolo della carbon footprint (quantità × fattore di emissione ÷ 1000) è eseguito dal database con precisione numerica a 4 decimali. Il bilancio di sostenibilità si basa su cifre calcolate dal motore ACID di PostgreSQL, non da JavaScript.

Generazione Automatica del Codice Progetto

Ogni nuovo engagement ottiene automaticamente un codice univoco nel formato `ESG-{anno}-{SIGLA}-{NNN}` tramite un trigger BEFORE INSERT (migration 00008). Il meccanismo è **race-safe**: usa `pg_advisory_xact_lock(hashtextextended(v_prefix, 0))` per serializzare eventuali inserimenti concorrenti con lo stesso prefisso anno+sigla, prevenendo duplicati anche in condizioni di carico elevato. Due inserimenti con prefissi diversi procedono in parallelo senza bloccarsi.

§05 Sicurezza: Architettura a Difesa Stratificata

Principio Fondante: Zero Trust Database

ESG Nexus non si affida alla logica applicativa per proteggere i dati. La sicurezza è implementata nel database stesso, al livello più basso possibile, usando **Row Level Security (RLS)** di PostgreSQL. Anche se il codice frontend fosse completamente compromesso o sostituito, i dati rimarrebbero inaccessibili perché la protezione vive a livello di ogni singola riga del database.

Row Level Security: 17 Tabelle, Zero Deleghe

Tutte le 17 tabelle dati hanno RLS abilitato con policy esplicite. Nessuna tabella è "dimenticata" o coperta da commenti generici — ogni policy è scritta esplicitamente nel file `00002_rls_policies.sql`.

Pattern 1 — Proprietà diretta (tabelle con `user_id`):

```
CREATE POLICY owner_all ON clienti FOR ALL
  USING (user_id = auth.uid())
  WITH CHECK (user_id = auth.uid());
```

Il consulente può vedere e modificare solo i propri clienti. L'uguaglianza `user_id = auth.uid()` è valutata da PostgreSQL per ogni singola riga, per ogni singola query.

Pattern 2 — Proprietà indiretta (tabelle figlie via JOIN):

```
CREATE POLICY via_eng ON form_data FOR ALL
  USING (EXISTS (
    SELECT 1 FROM engagements
    WHERE engagements.id = form_data.engagement_id
      AND engagements.user_id = auth.uid()
  ))
```

Un `form_data` è accessibile solo se l'engagement padre appartiene all'utente autenticato. La catena di controllo è: `auth.uid() → engagements → form_data`. Non esiste modo di accedere a `form_data` di un altro utente anche conoscendone l'UUID.

Pattern 3 — Cataloghi read-only:

```
CREATE POLICY catalog_read ON catalogo_iro FOR SELECT
  USING (auth.role() = 'authenticated');
```

■ Gestione Sicura delle Credenziali

- La **anon key** (chiave pubblica Supabase) è l'unica chiave presente nel codice frontend e nei build artifacts. È progettata per essere pubblica — la sicurezza non dipende dalla sua segretezza.
- La **service_role key** (accesso admin senza RLS) non è mai presente nel codice sorgente né nelle migrazioni. È iniettata come variabile d'ambiente solo nelle Edge Functions a runtime, e in Supabase Vault per il webhook del database.
- Il **JWT dell'utente** (access token) è gestito interamente da `@supabase/supabase-js` con refresh automatico. Il codice non lo legge né lo manipola direttamente.

Sicurezza delle HTTP Response Headers (Vercel)

Ogni risposta HTTP di ESG Nexus porta un set completo di security headers configurati in `vercel.json`:

HEADER	VALORE	PROTEZIONE
<code>X-Frame-Options</code>	DENY	Previene clickjacking / iframe embedding
<code>X-Content-Type-Options</code>	nosniff	Previene MIME type sniffing
<code>Referrer-Policy</code>	strict-origin-when-cross-origin	Limita leak di URL nelle richieste cross-origin
<code>Strict-Transport-Security</code>	max-age=63072000; includeSubDomains; preload	HTTPS obbligatorio per 2 anni, inclusi sottodomini
<code>Permissions-Policy</code>	camera=(), microphone=(), geolocation=()	Disabilita API hardware non necessarie
<code>Content-Security-Policy</code>	default-src 'self'; connect-src *.supabase.co	Whitelist esplicita delle sorgenti autorizzate
<code>Cache-Control</code> (assets)	public, max-age=31536000, immutable	Performance massima per asset statici con hash

Autenticazione Multi-Layer

L'autenticazione è gestita da **Supabase Auth**, un servizio battle-tested costruito su GoTrue. Il flusso è:

1. L'utente inserisce email + password
2. Supabase verifica le credenziali e rilascia un JWT firmato con HS256 (chiave del progetto)
3. Il JWT contiene: `sub` (user UUID), `role = authenticated`, `exp` (scadenza 1h)
4. Ogni richiesta API include il JWT nell'header `Authorization: Bearer <token>`
5. Il layer PostgREST di Supabase valida il JWT e imposta `auth.uid() = UUID` dell'utente
6. PostgreSQL usa `auth.uid()` per valutare le policy RLS su ogni riga

ARCHITETTURA STATELESS

Nessuna sessione server-side, nessun cookie, nessun stato condiviso. Il sistema è completamente stateless e scalabile orizzontalmente.

§06 Automazione: Edge Functions e Trigger in Background

Edge Function 1: Calcolo Progresso in Tempo Reale

`compute-engagement-progress` è una Edge Function Deno distribuita globalmente su Supabase, invocata automaticamente da un Database Webhook ogni volta che un record `form_data` viene inserito o aggiornato.

Il meccanismo di deduplicazione webhook (fix LOG-003) garantisce che, anche se il sistema riceve più notifiche per la stessa scrittura (retry, batching), il calcolo viene eseguito una sola volta — la funzione confronta il timestamp del payload con il record più recente nel database e salta l'esecuzione se esiste una versione più aggiornata.

Il risultato: il progresso dell'engagement nella dashboard si aggiorna in tempo reale, senza polling, senza interventi manuali.

Edge Function 2: Gestione Automatica delle Scadenze

`check-deadlines` viene eseguita automaticamente ogni 6 ore tramite `pg_cron` (PostgreSQL scheduling extension). Ad ogni esecuzione:

- Marca come scadute tutte le scadenze in stato `pending` con `data_scadenza < oggi`
- Genera notifiche per tutte le scadenze nei prossimi 7 giorni non ancora notificate, scrivendo eventi nel registro `eventi_log`
- Stampa l'orario di notifica (`notificata_at`) per garantire idempotenza — la stessa scadenza non viene mai notificata due volte

Risultato: il consulente non dimentica mai una deadline critica. Il sistema la gestisce autonomamente.

Edge Function 3: Generazione Codici Progetto

`generate-project-code` calcola il prossimo codice progetto disponibile nel formato `ESG-{anno}-{SIGLA}-{NNN}`, bypassando il RLS per leggere tutti gli engagement con quel prefisso (indipendentemente dal proprietario), garantendo unicità globale del codice. Questo stesso algoritmo è replicato anche nel trigger BEFORE INSERT 00008 come layer di sicurezza aggiuntivo.

Edge Function 4: Notifiche Email

`send-notification-email` è integrata con **Resend** (servizio email transazionale) per l'invio di notifiche via email. L'API key di Resend è conservata come segreto in Supabase Vault — mai nel codice sorgente.

Trigger Automatici Database (13 trigger attivi)

Il database gestisce automaticamente gli `updated_at` di tutte le tabelle modificabili tramite il trigger `set_updated_at()` — un pattern standard che garantisce che ogni record abbia sempre un timestamp di modifica accurato, indipendentemente da quale API o strumento ha effettuato la modifica.

Un trigger separato (`handle_new_user`) crea automaticamente il profilo utente (`users_profile`) alla registrazione, evitando race condition tra la creazione dell'utente Auth e la creazione del profilo applicativo.

§07 Persistenza dei Dati del Form: La Soluzione al Problema della Perdita Dati

Il Problema

Nei sistemi web tradizionali, i dati inseriti in un form vengono persi se l'utente chiude accidentalmente il browser, naviga via, o la connessione si interrompe. Per un consulente che inserisce dati critici di un bilancio di sostenibilità, questo è inaccettabile.

La Soluzione: useFormData Hook (Triple-Audited)

ESG Nexus implementa un meccanismo di salvataggio automatico avanzato attraverso l'hook `useFormData`, che è stato sottoposto a tre cicli di audit indipendenti e ha superato tutti i test. Il meccanismo ha **5 livelli di protezione**:

L1 DEBOUNCE INTELLIGENTE

Ogni modifica a un campo aggiorna immediatamente la UI locale e schedula una scrittura su Supabase dopo 1 secondo di inattività. L'utente vede sempre i propri dati, senza delay percepibili.

L2 STATO IN USEREF (ZERO STALE CLOSURE)

Il valore corrente dei dati è mantenuto in un ref (non nello state React), garantendo che il debounce legga sempre la versione più recente dei dati, mai una closure stale del passato.

L3 SERIALIZZAZIONE DEI FLUSH CONCORRENTI (INFLIGHTREF)

Se due salvataggi vengono schedulati in rapida successione, il secondo aspetta il completamento del primo prima di procedere. Questo previene la condizione di race in cui una scrittura più vecchia sovrascrive dati più recenti (il campo `data` in `form_data` è un JSONB che viene sostituito interamente ad ogni upsert).

L4 FLUSH SU NAVIGAZIONE (BEFOREUNLOAD)

Quando l'utente chiude o ricarica la pagina, il browser chiama l'event handler `beforeunload` che esegue una scrittura finale con `flushToSupabase()`.

L5 FLUSH SU SMONTAGGIO DEL COMPONENTE (REACT CLEANUP + KEEPALIVE)

Quando React smonta il componente (navigazione tra tab), il cleanup dell'effect usa `fetch` con `{ keepalive: true }` — una API browser che garantisce che la richiesta HTTP venga completata anche dopo che la pagina è stata smontata, a differenza delle fetch normali che verrebbero cancellate.

NOTA IMPLEMENTATIVA

Il sistema non usa `navigator.sendBeacon` (che non supporta header di autenticazione personalizzati) né `useBlocker` di React Router (che non funziona con operazioni asincrone). Queste non-scelte sono deliberate e documentate.

§08

Analytics e Business Intelligence

■ Quattro Dimensioni di Analisi

La sezione Analytics di ESG Nexus aggrega i dati di tutti gli engagement in visualizzazioni interattive organizzate in 4 tab:

1. PORTFOLIO & FATTURATO

- Andamento fatturato annuo (bar chart, asse Y in €k, fonte: `v_fatturato_annuo` view)
- Distribuzione engagement per standard ESG (donut chart: GRI / CSRD_ESRS / Entrambi)
- Tabella completa engagement con cliente, settore, codice progetto, stato, progresso

2. PERFORMANCE ESG

- Emissioni GHG aggregate per Scope 1/2/3 (bar chart, unità tCO₂e, fonte: `v_ghg_totali`)
- Stato validazione KPI per area E/S/G (stacked bar: compilati vs da compilare, fonte: `v_kpi_dashboard`)
- Distribuzione settori clienti (donut chart)

3. OPERATIVO & PROCESSI

- Progresso per engagement (barre orizzontali, colore verde/teal/amber in base alla percentuale)
- Distribuzione rischi per fascia di score (Critico ≥20 / Alto 15-19 / Medio 8-14 / Basso <8)

4. BENCHMARK CLIENTI

- Confronto dimensionale dei clienti per standard ESG adottato e stato avanzamento

■ Views PostgreSQL: Aggregazioni Server-Side

Le aggregazioni pesanti sono calcolate da **4 Views ottimizzate** nel database, non nel codice JavaScript:

VIEW	CONTENUTO
<code>v_engagement_summary</code>	join engagement + clienti + form_data + scadenze con COUNT FILTER per form completati e scadenze imminenti (14gg)
<code>v_ghg_totali</code>	SUM(co2e_tonnellate) per engagement e scope
<code>v_kpi_dashboard</code>	COUNT KPI compilati vs target raggiunti per area
<code>v_fatturato_annuo</code>	SUM(importo_cent)/100 per anno, con conteggio engagement

Il browser non scarica mai raw data per aggregarli in JavaScript — riceve direttamente i totali pre-calcolati, con query times dell'ordine dei millisecondi anche con grandi volumi di dati.

Realtime con WebSocket

La Dashboard è l'unica pagina dell'applicazione che non richiede refresh per aggiornarsi. Usa le **Realtime Subscriptions** di Supabase (WebSocket su `wss://`) per ricevere notifiche push dal database sulle tabelle `engagements` , `eventi_log` e `azioni_giorno` . Quando un dato cambia — anche da un altro dispositivo — la dashboard si aggiorna automaticamente in meno di un secondo.

Optimistic Updates per la Task List

Il completamento di un task nella sezione "La mia giornata" usa un **optimistic update pattern**: la checkbox si segna visivamente completata immediatamente al click, senza attendere la conferma del server. Se la scrittura fallisce, la UI torna automaticamente allo stato precedente (rollback). Questo crea un'esperienza utente istantanea e fluida, tipica delle applicazioni native.

Indicatori di Sincronizzazione

Un `SyncIndicator` globale nell'header mostra in tempo reale lo stato di sincronizzazione con il backend: "*Sincronizzazione...*" con animazione (durante scritture/letture attive); "*Tutto salvato*" con spunta verde (per 2,5 secondi dopo il completamento); un badge separato "*Offline*" appare automaticamente se la connessione internet viene persa.

§10 Gestione Rischi: Il Registro Integrato

■ Score Automatico 5×5

Il registro rischi implementa una matrice standard probabilità × impatto (scala 1-5 ciascuna). Lo score è una colonna

`GENERATED ALWAYS AS (probabilita * impatto) STORED` — calcolata da PostgreSQL, mai dalla UI. Questo garantisce:

- Impossibilità di inserire score inconsistenti (es. probabilità 2 × impatto 3 ≠ score 7)
- Classificazione automatica: **Basso** (<8), **Medio** (8-14), **Alto** (15-19), **Critico** (≥20)
- La dashboard mostra in evidenza i rischi critici per invitare il consulente all'azione

■ Visibilità Trasversale

Nella pagina di dettaglio di un engagement, i 5 rischi più critici (score ≥ 12) sono esposti in una tabella dedicata nella tab Panoramica, con colori che segnalano la gravità. Il consulente non deve navigare in sezioni separate per identificare i problemi più urgenti.

§11 Resilienza e Continuità del Dato

■ Backup Automatico Gestito da Supabase

ESG Nexus opera su Supabase Cloud (hosted PostgreSQL managed), che include:

- **Point-in-Time Recovery (PITR)** — backup continuo con granularità al secondo, disponibile nei piani Pro e superiori
- **Backup giornalieri automatici** — snapshot completo del database ogni 24 ore
- **Replica geografica** — il dato è replicato su almeno due availability zone AWS
- **Uptime SLA** — Supabase garantisce 99.9% uptime sull'infrastruttura database

Il consulente non deve configurare, monitorare o pagare separatamente alcun sistema di backup. È incluso nell'infrastruttura.

■ Deployment Multi-Layer con Zero Downtime

Il deploy di ESG Nexus avviene su **Vercel** — una CDN globale con PoP (Point of Presence) in Europa, America e Asia. Ogni deploy:

- Genera una build Vite ottimizzata con tree-shaking e code splitting (bundle iniziale ~555KB, da 1.58MB prima dell'ottimizzazione lazy loading)
- Viene distribuito globalmente su tutti i PoP Vercel in pochi secondi
- Supporta preview deployments su ogni pull request, con URL univoco per testing
- Il rollback a una versione precedente richiede un click nell'interfaccia Vercel

Le migrazioni database sono versionali e irreversibili (forward-only), seguendo le best practice di schema evolution. Ogni migrazione è numerata (`00001_` → `00008_`) e applicata in sequenza, garantendo che l'ambiente di staging e quello di produzione siano sempre allineati.

■ Integrità Transazionale ACID

Tutte le scritture su PostgreSQL sono ACID-compliant (Atomicity, Consistency, Isolation, Durability). Questo significa:

- Un upsert su `form_data` non può essere parzialmente applicato — o riesce completamente o fallisce senza tracce
- Due consulenti diversi non possono mai vedere dati dell'uno nell'ambiente dell'altro (Isolation + RLS)
- Un dato scritto è permanente fino a modifica esplicita (Durability)

§12 Qualità del Codice e Processo di Sviluppo

Validazione Dati con Zod

Prima di qualsiasi scrittura su Supabase, tutti i dati passano attraverso schemi Zod che validano:

- `clienteSchema` — controlla formato P.IVA, codice fiscale, nazione ISO alpha-2, URL sito web
- `engagementSchema` — controlla standard (enum GRI/CSRD_ESRS/ENTRAMBI), date in formato YYYY-MM-DD, anno tra 2015 e 2050
- `rischioSchema` — controlla che probabilità e impatto siano interi 1–5
- `scadenzaSchema` — controlla formato data_scadenza

Gli errori di validazione vengono mostrati inline accanto al campo problematico, non come popup generici. Il DB non viene mai chiamato con dati non validi.

Pipeline CI/CD Automatizzata

Ogni push al repository GitHub attiva automaticamente la pipeline GitHub Actions:

- **Lint** — eslint con plugin react, react-hooks, unused-imports (zero warning tollerati in CI)
- **Test** — vitest run con 5 test su 3 file critici (`useClienti` , `useFormData` , `useEngagements`) usando `@testing-library/react` e MSW per mock HTTP
- **Build** — vite build con placeholder delle env vars, verifica che il bundle si compili senza errori

La pipeline usa **concurrency cancellation**: se viene pushato un nuovo commit mentre una pipeline è in esecuzione, quella vecchia viene annullata — nessuna coda di build obsolete.

Il bundle ottimizzato usa lazy loading con `React.lazy()` + `Suspense` per tutte le 9 pagine principali. Al caricamento iniziale, il browser scarica solo il codice necessario per la pagina corrente. Le altre pagine vengono caricate on-demand al primo accesso.

ErrorBoundary Globale

Un `ErrorBoundary` React avvolge tutta l'applicazione. Se un componente genera un'eccezione JavaScript non gestita, l'utente vede una pagina di errore controllata con opzione di reset o reload — mai una schermata bianca. Gli errori vengono loggati in console per debugging, senza esporre stack trace all'utente finale.

§13 Funzionalità di Stampa e Archiviazione

ESG Nexus include un sistema di stampa professionale direttamente dal browser. Ogni form operativo può essere stampato o salvato come PDF tramite il dialog nativo del browser (Ctrl+P / Cmd+P o il pulsante "Stampa / Salva PDF").

Il sistema applica automaticamente un CSS `@media print` dedicato che:

- Imposta il formato A4 con margini 18mm×14mm
- Rimuove dalla stampa: sidebar, topbar, tab di navigazione, bottoni azione, indicatori di sincronizzazione
- Forza il tema chiaro anche se l'utente sta usando il dark mode
- Applica regole `break-inside: avoid` per evitare di spezzare sezioni a metà pagina

- Elimina bordi e shadow decorativi per un aspetto pulito su carta

Il risultato è un documento PDF professionale, pronto per archiviazione o consegna al cliente, generato direttamente dai dati live del sistema senza esportazioni intermedie.

§14 Glossario Tecnico (Per il Cliente)

TERMINE	SIGNIFICATO PRATICO
RLS <i>Row Level Security</i>	Sistema che garantisce che ogni utente veda solo i propri dati, applicato a livello database — non dipende dal codice dell'applicazione
Edge Function	Codice server che gira automaticamente in risposta a eventi (es. salvataggio di un form), senza bisogno di un server dedicato
JWT <i>JSON Web Token</i>	Credenziale cifrata che identifica l'utente per ogni richiesta al sistema, valida per 1 ora poi rinnovata automaticamente
JSONB	Formato di archiviazione dei dati del form in PostgreSQL — flessibile come JSON, ma indicizzabile e queryable con performance native
pg_cron	Estensione PostgreSQL che esegue job pianificati (come il controllo delle scadenze ogni 6 ore) direttamente nel database
PITR <i>Point-in-Time Recovery</i>	Possibilità di ripristinare il database a qualsiasi momento nel passato, con granularità al secondo
WebSocket	Connessione persistente tra browser e server che permette aggiornamenti in tempo reale senza refresh della pagina
Zod	Libreria che verifica che i dati abbiano il formato corretto prima di salvarli — come un controllo qualità automatico
ACID	Proprietà fondamentali di un database affidabile: ogni scrittura è atomica, consistente, isolata e permanente

§15 Evoluzioni Future del Sistema

Questa sezione documenta le direzioni di sviluppo identificate per le versioni successive di ESG Nexus. Ogni punto descrive una funzionalità non ancora implementata, il problema che risolverebbe e come si integrerebbe nell'architettura esistente.

14.1 — Generazione Automatica del Report di Engagement (PDF Strutturato)

Cosa sarebbe: Un motore di generazione PDF server-side che, al termine di un engagement, produce automaticamente un documento di 30–50 pagine contenente tutti i dati raccolti nelle 8 procedure: anagrafica cliente, analisi di materialità, matrice IRO, dati GHG per scope, KPI E/S/G con valori e target, registro rischi, milestone e stato di avanzamento.

Come funzionerebbe: Utilizzerebbe una libreria come `@react-pdf/renderer` (React → PDF) o Puppeteer (HTML → PDF headless) eseguita in una Edge Function Deno. Il documento sarebbe generato on-demand dal consulente con un click, e archiviato automaticamente nel bucket Storage di Supabase associato all'engagement.

Cosa risolverebbe: Attualmente il consulente deve assemblare manualmente il report finale dai dati distribuiti nei vari form. La generazione automatica elimina questa fase e garantisce che il documento rifletta sempre i dati live del sistema al momento della generazione.

14.2 — Firma Digitale dei Documenti

Cosa sarebbe: Integrazione con un servizio di firma elettronica (DocuSign, Yousign, o servizi equivalenti conformi eIDAS) per la firma remota di documenti chiave: contratti di incarico, offerte commerciali, lettere di attestazione del bilancio.

Come funzionerebbe: Il consulente carica o genera un documento da firmare, il sistema invia via email al destinatario un link di firma sicuro. Lo stato della firma (inviato / firmato / scaduto) è tracciato in un'apposita tabella del database e visibile nella dashboard. Il documento firmato viene archiviato nel bucket Storage dell'engagement.

Cosa risolverebbe: Attualmente i documenti devono essere stampati, firmati fisicamente e riscansionati. L'integrazione elimina questa frizione operativa e mantiene tutto il ciclo documentale all'interno del sistema, con piena tracciabilità.

14.3 — Portale Cliente con Accesso Esterno

Cosa sarebbe: Un'area riservata accessibile dal cliente finale (l'azienda che commissiona il bilancio ESG), separata dall'interfaccia del consulente. Il cliente potrebbe accedere a una vista di sola lettura del proprio engagement: stato avanzamento, scadenze, documenti condivisi, KPI inseriti.

Come funzionerebbe: Tecnicamente richiederebbe un secondo ruolo applicativo (`cliente_viewer`) con policy RLS dedicate che limitano la visibilità ai soli dati dell'engagement di competenza. Il portale potrebbe essere un sottodominio separato (`cliente.esg-nexus.app`) con branding personalizzabile per ogni studio di consulenza.

Cosa risolverebbe: Attualmente il cliente riceve aggiornamenti via email o call periodiche. Il portale darebbe accesso diretto e controllato alle informazioni rilevanti, riducendo il carico comunicativo sul consulente e aumentando la trasparenza del processo.

Cosa sarebbe: Un layer AI integrato nei form operativi che, a partire da documenti caricati dal consulente (bilanci precedenti, relazioni annuali, documenti gestionali del cliente in PDF), suggerisce automaticamente i valori da inserire nei campi del form.

Come funzionerebbe: I documenti vengono indicizzati tramite un modello di embedding e archiviati in un vector store. Quando il consulente apre un form, una chiamata all'API di un modello LLM (es. Claude API di Anthropic) estrae dal contesto documentale le informazioni pertinenti e le propone come pre-compilazione. Il consulente rivede, corregge e conferma — l'AI non scrive autonomamente nel database.

Cosa risolverebbe: La raccolta dati è la fase più time-intensive di un engagement ESG. Molte informazioni esistono già in documenti aziendali non strutturati. L'assistenza AI riduce il tempo di compilazione e il rischio di dimenticare dati disponibili.

■ 14.5 — Benchmark di Settore

Cosa sarebbe: Un modulo di analisi comparativa che mette a confronto i KPI e i dati GHG di un cliente con le medie aggregate (anonimizzate) di altri clienti dello stesso settore ATECO presenti nel sistema, o con dataset pubblici di benchmark di settore (GRI, CDP, CSRD baseline).

Come funzionerebbe: Le aggregazioni benchmark sarebbero calcolate da una View PostgreSQL o da un job pg_cron schedulato, aggregando i dati anonimizzati per settore e standard. Il consulente vedrebbe nella pagina Analytics una tab aggiuntiva "Benchmark" con i posizionamenti relativi del cliente.

Cosa risolverebbe: Oggi il consulente non ha un riferimento quantitativo interno per capire se i KPI di un cliente sono nella norma per il suo settore. Il benchmark rende questa valutazione immediata e basata sui dati reali del portafoglio.

■ 14.6 — Monitoraggio Automatico delle Scadenze Normative

Cosa sarebbe: Un calendario integrato delle scadenze di legge legate agli standard ESG adottati dall'engagement (date di pubblicazione obbligatoria CSRD, scadenze ESRS, deadline GRI), aggiornato automaticamente quando la normativa viene modificata.

Come funzionerebbe: Un database di scadenze normative per framework e tipologia di impresa (dimensione, settore, quotazione) sarebbe mantenuto come catalogo separato nel sistema. Alla creazione di un engagement, il sistema popolava automaticamente le scadenze normative rilevanti nella tabella `scadenze`, con priorità impostata a critica.

Cosa risolverebbe: Le scadenze normative CSRD/ESRS variano per tipologia di impresa e cambiano con gli aggiornamenti legislativi. Attualmente il consulente deve tracciarle manualmente. L'automazione riduce il rischio di non conformità per mancata awareness delle deadline.

■ 14.7 — Raccolta Dati da Stakeholder Esterni via Questionario

Cosa sarebbe: Un sistema per inviare questionari strutturati a soggetti esterni (fornitori, dipendenti, comunità locali) per raccogliere dati rilevanti per l'analisi di materialità o per la rendicontazione sociale. Le risposte confluiscono direttamente nei form dell'engagement.

Come funzionerebbe: Il consulente configura un questionario selezionando un sottoinsieme di campi da un form esistente. Il sistema genera un link univoco (token monouso, scadenza configurabile) che il destinatario apre senza necessità di autenticazione. Le risposte vengono scritte in una tabella staging separata e il consulente le approva prima dell'importazione definitiva nel form.

Cosa risolverebbe: La raccolta dati da soggetti terzi (es. fornitori per Scope 3, dipendenti per dati sociali) avviene oggi via email o Excel inviati manualmente. Il sistema strutturato garantisce la tracciabilità delle risposte, riduce i formati non standard e integra i dati direttamente nell'engagement.

■ 14.8 — Dashboard Pubblica del Bilancio

Cosa sarebbe: Una pagina web pubblica, accessibile senza login, che mostra una selezione dei KPI e degli indicatori ESG di un cliente — una versione sintetica e visivamente curata del bilancio di sostenibilità, pensata per essere linkata dal sito istituzionale del cliente.

Come funzionerebbe: Il consulente attiva la pubblicazione per un engagement completato, seleziona quali KPI e dati rendere visibili, e il sistema genera una pagina con URL dedicato (`/public/engagement/{slug}`). I dati mostrati sono

un subset read-only estratto dall'engagement, senza dati commerciali né informazioni riservate. La pagina è aggiornabile ogni volta che il consulente decide di pubblicare una nuova versione.

Cosa risolverebbe: I bilanci di sostenibilità sono documenti PDF statici che richiedono download. Una dashboard pubblica rende i dati ESG più accessibili agli stakeholder (investitori, clienti, media) e aumenta la visibilità del lavoro di rendicontazione.

■ 14.9 — Integrazione con Sistemi Gestionali Aziendali

Cosa sarebbe: Connettori bidirezionali con i principali ERP e sistemi contabili aziendali (SAP, Zucchetti, TeamSystem, Fatture in Cloud) per importare automaticamente dati quantitativi rilevanti per il bilancio ESG: consumi energetici, dati HR, fatturato, costi per categoria.

Come funzionerebbe: Ogni connettore sarebbe implementato come Edge Function separata, invocabile manualmente o su schedule. I dati importati vengono mappati sui campi dei form corrispondenti e proposti al consulente per validazione prima della scrittura definitiva. Le credenziali dei sistemi terzi sarebbero archiviate in Supabase Vault.

Cosa risolverebbe: Una parte significativa dei dati necessari per la rendicontazione ESG esiste già nei sistemi gestionali del cliente. L'importazione manuale di questi dati nei form è time-consuming e soggetta a errori di trascrizione. L'integrazione automatizza il trasferimento e garantisce coerenza tra i dati gestionali e i dati ESG rendicontati.

Le funzionalità descritte in questa sezione non sono parte del sistema attuale. Rappresentano direzioni di sviluppo identificate sulla base dell'architettura esistente e delle esigenze operative osservate nel processo di consulenza ESG.
